

- Akansha Balhara
- 1/23/SET/BCS/433
- 6AIMLB1
- DATE: 02/04/2026
- EXPERIMENT: 10

```
!pip uninstall rsa -y
!pip install rsa
```

```
import os
os.kill(os.getpid(), 9)  # restart runtime automatically
```

```
Found existing installation: rsa 4.9.1
Uninstalling rsa-4.9.1:
  Successfully uninstalled rsa-4.9.1
Collecting rsa
  Downloading rsa-4.9.1-py3-none-any.whl.metadata (5.6 kB)
Requirement already satisfied: pyasn1>=0.1.3 in
/usr/local/lib/python3.12/dist-packages (from rsa) (0.6.3)
Downloading rsa-4.9.1-py3-none-any.whl (34 kB)
Installing collected packages: rsa
Successfully installed rsa-4.9.1
```

Iris Dataset

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Flatten, Dense
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from sklearn.preprocessing import StandardScaler
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Normalize features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Convert to "image" shape (2x2x1)
X = X.reshape(-1, 2, 2, 1)

# One-hot encode labels
y = to_categorical(y, 3)
```

```

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Build CNN model
model = Sequential([
    Conv2D(32, (2,2), activation='relu', input_shape=(2,2,1)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(3, activation='softmax')
])

# Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Summary
model.summary()

# Train (STORE HISTORY)
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=128,
    validation_split=0.1
)

# Evaluate
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# =====
# □ PLOT GRAPHS
# =====

# Accuracy Graph
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Accuracy vs Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train Accuracy', 'Validation Accuracy'])
plt.show()

# Loss Graph

```

```
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Loss vs Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(['Train Loss', 'Validation Loss'])
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

Model: "sequential_7"

Layer (type)	Output Shape	
Param #		
conv2d_13 (Conv2D)	(None, 1, 1, 32)	
160		
flatten_7 (Flatten)	(None, 32)	
0		
dense_14 (Dense)	(None, 128)	
4,224		
dense_15 (Dense)	(None, 3)	
387		

Total params: 4,771 (18.64 KB)

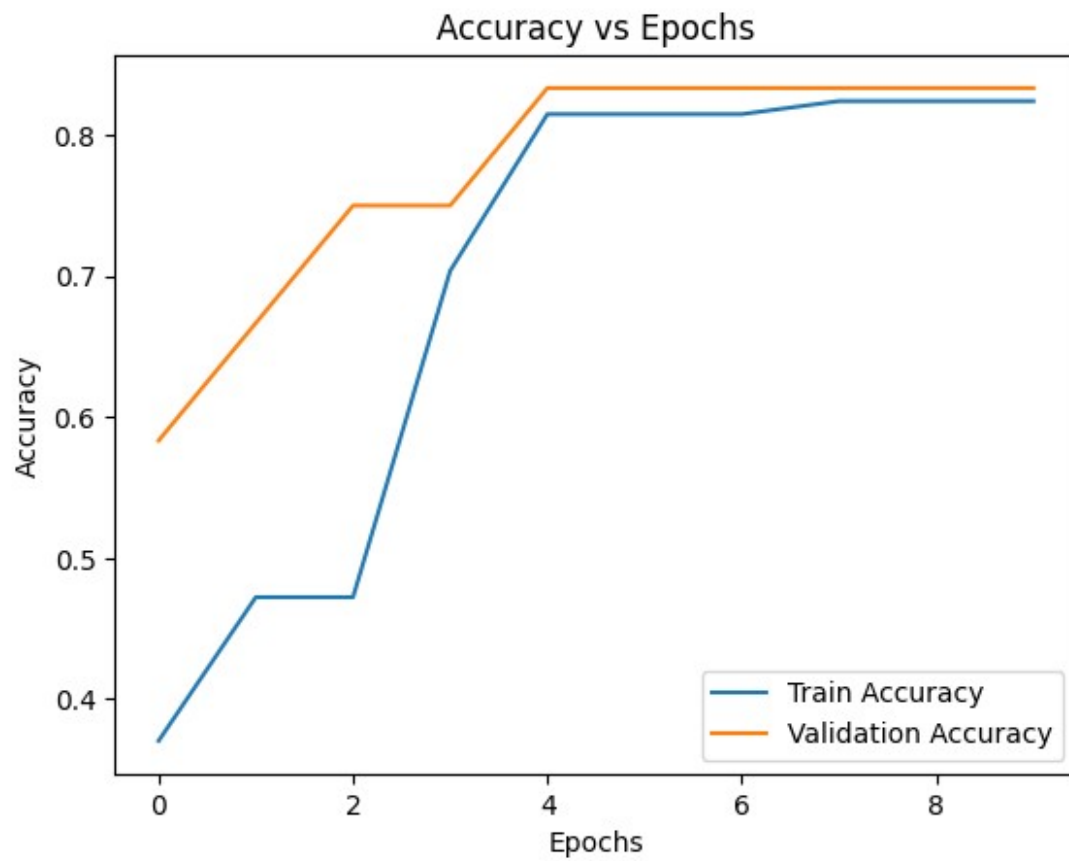
Trainable params: 4,771 (18.64 KB)

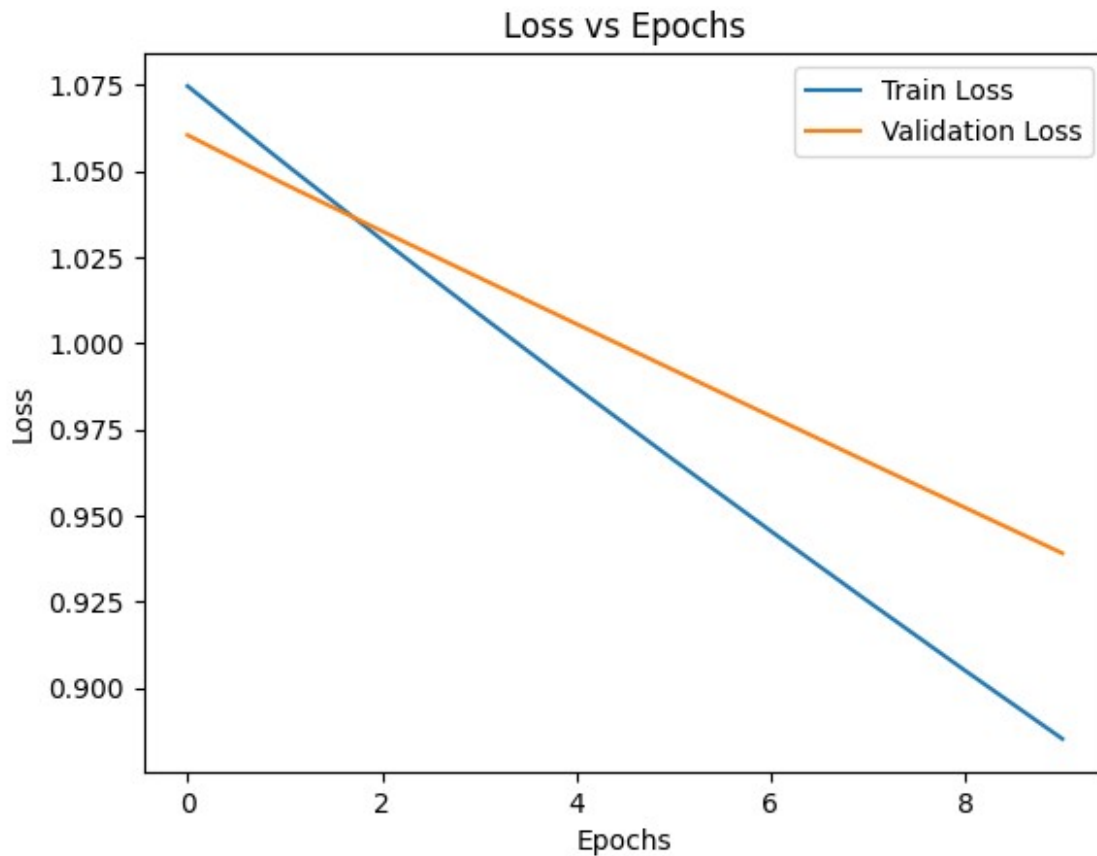
Non-trainable params: 0 (0.00 B)

Epoch 1/10

1/1 2s 2s/step - accuracy: 0.3704 - loss: 1.0747
- val_accuracy: 0.5833 - val_loss: 1.0605

```
Epoch 2/10
1/1 _____ 0s 99ms/step - accuracy: 0.4722 - loss:
1.0522 - val_accuracy: 0.6667 - val_loss: 1.0462
Epoch 3/10
1/1 _____ 0s 96ms/step - accuracy: 0.4722 - loss:
1.0301 - val_accuracy: 0.7500 - val_loss: 1.0326
Epoch 4/10
1/1 _____ 0s 98ms/step - accuracy: 0.7037 - loss:
1.0084 - val_accuracy: 0.7500 - val_loss: 1.0191
Epoch 5/10
1/1 _____ 0s 107ms/step - accuracy: 0.8148 - loss:
0.9871 - val_accuracy: 0.8333 - val_loss: 1.0056
Epoch 6/10
1/1 _____ 0s 156ms/step - accuracy: 0.8148 - loss:
0.9661 - val_accuracy: 0.8333 - val_loss: 0.9922
Epoch 7/10
1/1 _____ 0s 184ms/step - accuracy: 0.8148 - loss:
0.9455 - val_accuracy: 0.8333 - val_loss: 0.9788
Epoch 8/10
1/1 _____ 0s 140ms/step - accuracy: 0.8241 - loss:
0.9250 - val_accuracy: 0.8333 - val_loss: 0.9655
Epoch 9/10
1/1 _____ 0s 142ms/step - accuracy: 0.8241 - loss:
0.9049 - val_accuracy: 0.8333 - val_loss: 0.9522
Epoch 10/10
1/1 _____ 0s 297ms/step - accuracy: 0.8241 - loss:
0.8851 - val_accuracy: 0.8333 - val_loss: 0.9390
1/1 _____ 0s 100ms/step - accuracy: 0.9000 - loss:
0.8575
Test accuracy: 0.8999999761581421
```





Binary Classification using Dogs and Cats dataset

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
import zipfile
import os

# -----
# Dataset Path (Change if needed)
# -----
zip_path = "/content/archive.zip"
extraction_dir = '/content/dataset'

# Create the extraction directory if it doesn't exist
if not os.path.exists(extraction_dir):
    os.makedirs(extraction_dir)

# Extract the zip file
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
```

```

zip_ref.extractall(extraction_dir)

# Now, train_dir should point to the extracted dataset directory
train_dir = extraction_dir

# -----
# □ Image Preprocessing
# -----
datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

# Training Data
train_data = datagen.flow_from_directory(
    train_dir,
    target_size=(150, 150),
    batch_size=32,
    class_mode='binary',
    subset='training'
)

# Validation Data
val_data = datagen.flow_from_directory(
    train_dir,
    target_size=(150, 150),
    batch_size=32,
    class_mode='binary',
    subset='validation'
)

# -----
# □ DATASET PREVIEW (HEAD)
# -----
images, labels = next(train_data)

plt.figure(figsize=(10,5))
for i in range(5):
    plt.subplot(1,5,i+1)
    plt.imshow(images[i])
    plt.title(f"Label: {int(labels[i])}")
    plt.axis('off')

plt.suptitle("Dataset Preview (Head)")
plt.show()

# Show class labels
print("Class indices:", train_data.class_indices)

# -----

```

```

# □ CNN MODEL
# -----
model = Sequential([

    Conv2D(32, (3,3), activation='relu', input_shape=(150,150,3)),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Flatten(),
    Dense(128, activation='relu'),
    Dense(1, activation='sigmoid')

])

# Compile
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()

# -----
# □ TRAIN MODEL
# -----
history = model.fit(
    train_data,
    epochs=10,
    validation_data=val_data
)

# -----
# □ GRAPHS
# -----

# Accuracy Graph
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train Accuracy', 'Validation Accuracy'])
plt.show()

```

```
# Loss Graph
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(['Train Loss', 'Validation Loss'])
plt.show()
```

Found 800 images belonging to 2 classes.
Found 200 images belonging to 2 classes.

Dataset Preview (Head)



Class indices: {'cats_set': 0, 'dogs_set': 1}

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

Model: "sequential_5"

Layer (type) Param #	Output Shape
conv2d_9 (Conv2D) 896	(None, 148, 148, 32)

0	max_pooling2d_6 (MaxPooling2D)	(None, 74, 74, 32)
18,496	conv2d_10 (Conv2D)	(None, 72, 72, 64)
0	max_pooling2d_7 (MaxPooling2D)	(None, 36, 36, 64)
73,856	conv2d_11 (Conv2D)	(None, 34, 34, 128)
0	max_pooling2d_8 (MaxPooling2D)	(None, 17, 17, 128)
0	flatten_5 (Flatten)	(None, 36992)
4,735,104	dense_10 (Dense)	(None, 128)
129	dense_11 (Dense)	(None, 1)

Total params: 4,828,481 (18.42 MB)

Trainable params: 4,828,481 (18.42 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

25/25 ————— 44s 2s/step - accuracy: 0.5175 - loss: 0.7742 - val_accuracy: 0.5050 - val_loss: 0.6868

Epoch 2/10

25/25 ————— 41s 2s/step - accuracy: 0.5612 - loss: 0.6856 - val_accuracy: 0.5900 - val_loss: 0.6720

Epoch 3/10

25/25 ————— 47s 2s/step - accuracy: 0.6475 - loss: 0.6269 - val_accuracy: 0.5200 - val_loss: 0.7364

Epoch 4/10
25/25 ————— 42s 2s/step - accuracy: 0.6950 - loss: 0.5707 - val_accuracy: 0.5600 - val_loss: 0.6857
Epoch 5/10
25/25 ————— 42s 2s/step - accuracy: 0.7613 - loss: 0.4962 - val_accuracy: 0.6050 - val_loss: 0.6953
Epoch 6/10
25/25 ————— 42s 2s/step - accuracy: 0.8413 - loss: 0.3850 - val_accuracy: 0.5800 - val_loss: 0.8492
Epoch 7/10
25/25 ————— 43s 2s/step - accuracy: 0.8775 - loss: 0.2960 - val_accuracy: 0.5300 - val_loss: 0.9888
Epoch 8/10
25/25 ————— 82s 2s/step - accuracy: 0.9212 - loss: 0.2033 - val_accuracy: 0.5950 - val_loss: 1.0810
Epoch 9/10
25/25 ————— 42s 2s/step - accuracy: 0.9413 - loss: 0.1420 - val_accuracy: 0.5300 - val_loss: 1.6340
Epoch 10/10
25/25 ————— 45s 2s/step - accuracy: 0.9650 - loss: 0.1075 - val_accuracy: 0.6100 - val_loss: 1.7223

